# Kinect Lite Denoising: Project Guide

Oct 16, 2026 · @Sumanth G

## The project in one page

Kinect Lite cleans the depth images of a Kinect-type camera with simple spatial filters, before they are used for 3D tracking (ICP). This first phase answered one question: which filter removes the most depth error, and what does each one cost?

**What a depth camera gives you.** One distance per pixel, 640 x 480 pixels, 30 times a second. Some pixels are missing, and the rest are noisy: small jitter everywhere, and large errors at object edges.

**What we did.** We tested five filter families (median, invalid-aware median, Gaussian, bilateral, hole filling), each in several settings, on two datasets:

- **ICL-NUIM:** a rendered living room with simulated camera noise and the true depth of every pixel, so every filter gets an exact error score.
- **TUM fr3:** a real recording with an Asus Xtion (the same depth technology as the original Kinect). There is no true depth per pixel, but the true camera path is known.

**Outcome.** On ICL, the invalid-aware median cut the average depth error from 26.2 mm to 12.8 mm and removed two-thirds of the error at object edges, better than every Gaussian and bilateral setting. On real TUM data the averaging filters smooth flat surfaces far more, but they also create stray points between objects; which wins there is still open.

**What is next.** ICP: estimating the camera's movement from the depth images. TUM's true camera path will then score the filters on real data.

The theory (how depth cameras and filters work) is in the companion document *Kinect Lite: Depth Denoising*, and the full set of charts in *Kinect Lite: Denoising Results*. This guide explains what was done and where everything is.

## What was done, step by step

The work ran in eight steps; each one produced files you can open.

1. **First attempt (earlier).** Spatial and temporal filters were run together. The results were hard to read, so the scope was cut to spatial filters only and the code was rebuilt from scratch in `denoise/`.

2. **Data and evaluation checks** ( `denoise/audit.py` , `audit2.py` ). Before scoring any filter, we checked the data itself: units, missing pixels, where the error sits, how the noisy and true images line up. This found the edge spikes and the 1-pixel misalignment.
3. **Reproduction check** ( `experiments/reproduce_original.json` ). The new code was run with the earlier run's 12 settings and matched its numbers exactly, so nothing changed by accident.
4. **Tuning** ( `experiments/tune_*.json` ). Each filter family was run over a grid of settings on 31 tuning frames, with and without the 0.4 m gate. Gaussian was added on your request. The bilateral grid was extended twice because its best results sat at the edge of the range tried.
5. **Selection** ( `denoise/select_configs.py` ). For each family, the setting with the lowest error on the tuning frames was picked.
6. **Evaluation** ( `experiments/eval_selected.json` ). The picked settings were scored once on 30 different test frames, all on the same pixels.
7. **Real data** ( `experiments/tum_selected.json` ). The same settings were run on 50 TUM frames to see what they change on a real sensor.
8. **Pictures and write-ups.** Figures, 3D point-cloud views, the results folder, three documents, and this cleanup into a GitHub-ready repository.

The code was also rewritten once for readability; a re-run gave identical numbers.

## Key decisions and why

Each choice below was made to keep the comparison fair or the result honest.

| decision | why |
|---|---|
| Spatial filters only, one frame at a time | Temporal filtering mixed two effects and made results hard to read; ICP comes later anyway |
| Score on ICL, only describe TUM | Only ICL has the true depth of every pixel |
| Separate tuning and test frames (0, 50, 100 … vs 25, 75, 125 …) | Picking settings on the scored frames would flatter the result |
| Score flat areas and edges separately | 4 % of pixels carry 56 % of the error; one overall number hides what happens where |

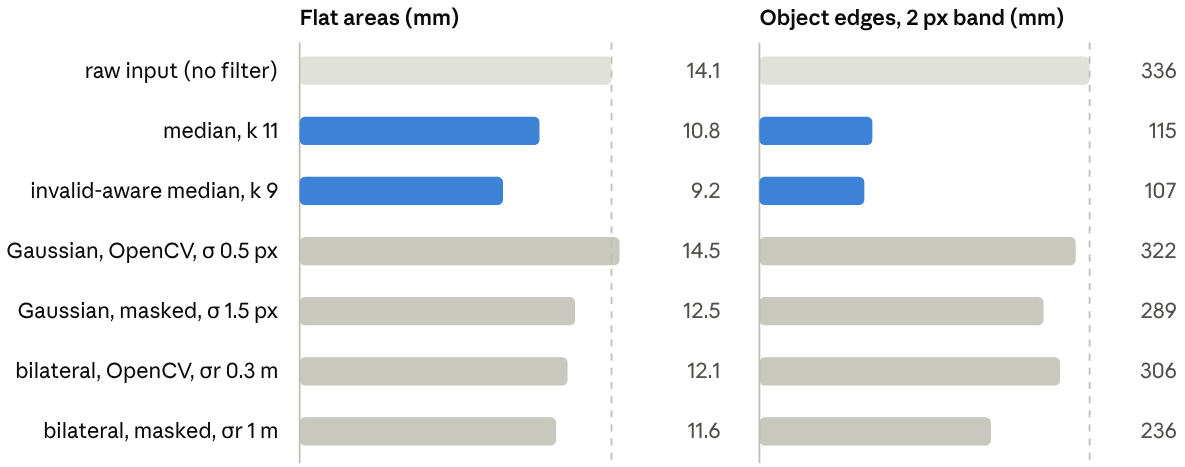
| decision | why |
|---|---|
| Same pixels for every filter (matched mask) | Filters that fill or delete pixels would otherwise be scored on different pixels |
| Edges taken from the true depth only | A filter must not be able to move the line it is judged against |
| Run every filter with and without the 0.4 m gate | A Kinect-type sensor cannot measure that close; those values are 0.5 % of pixels but 25 % of the error |
| OpenCV and masked versions of Gaussian and bilateral | OpenCV treats holes as 0 m, which biases results; the masked versions show the filter itself |
| Extend a grid when the best value sits on its edge | Otherwise a better setting just outside the grid would be missed |
| Report measurements, do not crown a winner | The best filter depends on the data and the use; ICL's noise is simulated |
| Never overwrite a finished run | Every number in the reports can be traced to a file |

## What we found

On ICL the medians win clearly, because most of the error is isolated spikes at object edges, which a median votes out and an average spreads.

**Medians remove two-thirds of the edge error; averaging filters 4-30 %**

Mean absolute error, 30 ICL test frames, one tuned setting per filter, 0.4 m gate. Blue = median filters.




Dashed lines = raw input. Bars start at 0 mm.

outputs/eval_selected/summary.csv · ICL frames 25:1509:50 (30 frames)

- **The noise is mostly edge spikes.** 70 % of bad edge pixels are values far too near the camera; in 3D they lie on lines from the camera to object edges. They likely come from how ICL simulates noise.
- **Medians remove spikes but shift outlines.** Edge error drops by two-thirds, but pixels on the wrong side of an edge rise from 8.6 % to about 20 %.
- **Missing-pixel handling matters as much as the filter.** OpenCV's Gaussian (holes counted as 0 m) is worse than raw on flat areas; the masked Gaussian with the gate is better.
- **The 0.4 m gate** helps averaging filters a lot on ICL, and does nothing on TUM, which has no depth below 0.4 m.
- **On real TUM data** the bilateral and masked Gaussian make flat surfaces 8-11x smoother but create stray points between objects. Without true depth, no winner can be named there yet.

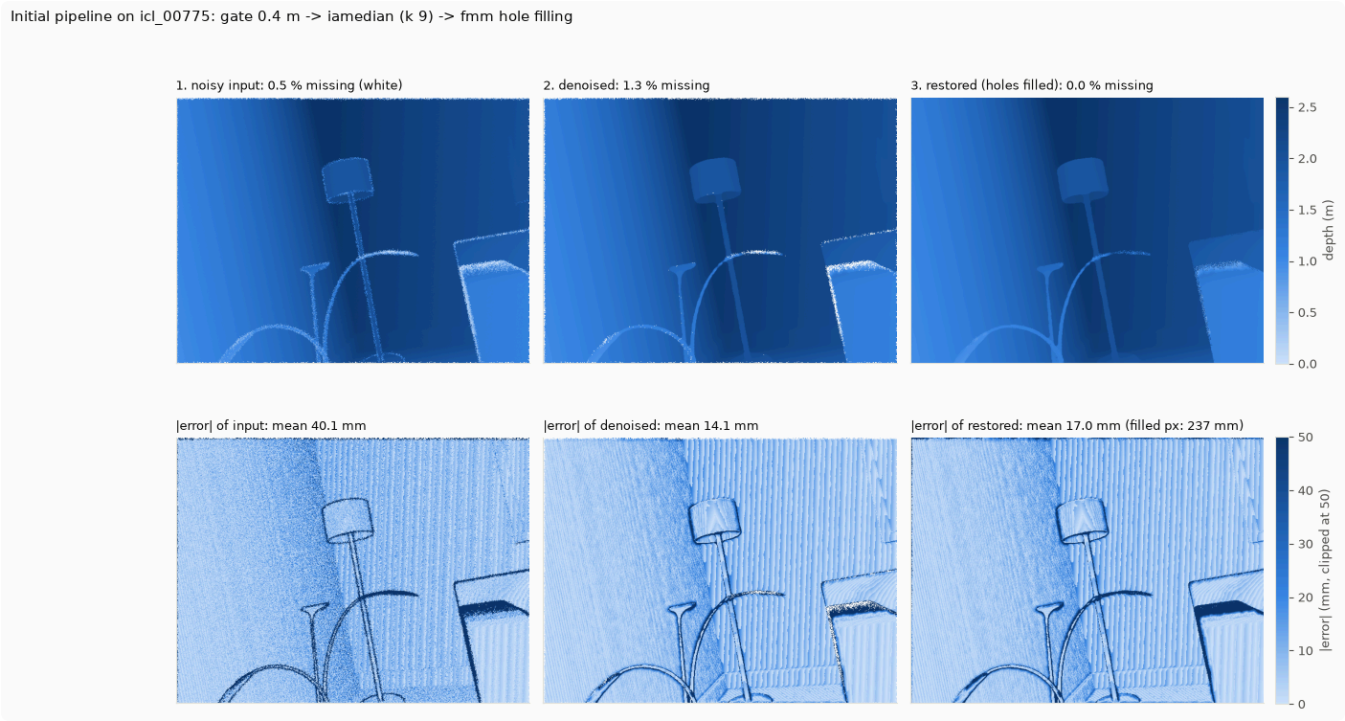
All charts are in *Kinect Lite: Denoising Results*.

## Initial pipeline: noisy to denoised image

The filters and hole filling are joined into one pipeline that turns a noisy depth image into a restored one with every pixel filled; `denoise/pipeline.py` runs it on any depth PNG.

1. **Noisy depth:** noise plus missing pixels.

2. **0.4 m gate:** depth closer than 0.4 m is treated as missing.
3. **Denoising filter:** invalid-aware median by default; median, Gaussian or bilateral on request.
4. **Denoised depth:** noise reduced, holes may remain.
5. **Hole filling:** FMM (Telea) inpainting by default; Navier-Stokes or region fill on request. Filled values are clipped to the depth range around each hole.
6. **Restored depth:** saved as a 16-bit PNG in the dataset format.




On ICL frame 775, denoising cuts the error from 40.1 mm to 14.1 mm; filling every hole raises it to 17.0 mm overall, because the filled pixels average 237 mm off.

| hole filling after the invalid-aware median (30 ICL test frames) | pixels filled | mean error of filled pixels |
|---|---|---|
| region fill (small interior holes only) | 1,390 | 60 mm |
| FMM, holes up to 1,024 px | 44,026 | 74 mm |
| FMM, all holes | 49,328 | 130 mm |
| Navier-Stokes, all holes | 49,328 | 33 mm |

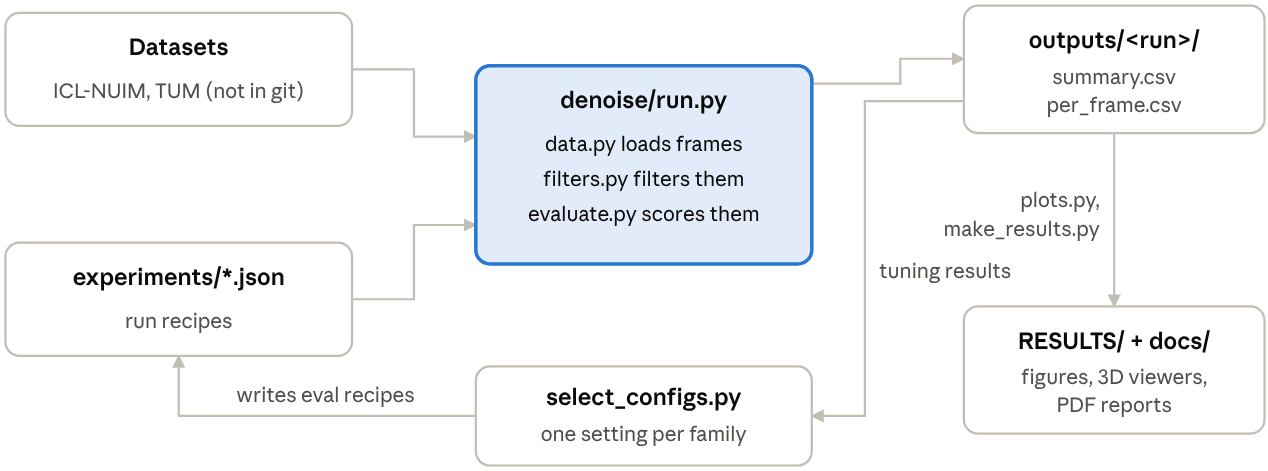
Most filled pixels are values the gate removed or holes on object edges, the hardest places to guess. Navier-Stokes filled the same holes about four times more accurately than FMM, and on TUM it kept flat areas smooth (roughness 2.34 mm vs 5.01 mm with FMM). FMM stays the default as planned; `--fill ns` switches.

```
python denoise/pipeline.py --input path/to/depth.png
python denoise/pipeline.py --icl 775 --figure
python denoise/pipeline.py --tum 1200 --fill ns --figure
```

## The repository

Everything runs through one script: `denoise/run.py` reads a recipe, filters the frames, scores them and writes the numbers to `outputs/`.

**Recipes and data go in, numbers come out, tuning results pick the next recipe**




how the repository fits together · 6 parts

| folder | what is in it |
|---|---|
| `denoise/` | all code; the top-level `README.md` lists what each file does |
| `experiments/` | run recipes, explained in `experiments/README.md` |
| `outputs/` | numbers of every run (`summary.csv` first), the data checks in `audit/`, all figures in `figures/` |

| folder | what is in it |
|---|---|
| `RESULTS/` | start here: numbered figures, one table, two 3D viewers, a README |
| `docs/` | findings, data audit, experiment plan, references |
| `docs/reports/` | the three documents as PDF and Word |
| `ICL NUIM/`, `RGB TUM/` | the datasets, kept out of git |

Also kept on disk but out of git: `.venv/` (Python packages), your `parsing.py` and progress-guide PDF, the published ICL paper, and the working checkpoint file.

## How to set up and run it

Three commands set it up; one command runs any recipe.

**Setup** (Windows, from the project folder):

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

Put the datasets in `ICL NUIM/` and `RGB TUM/` as listed in `README.md`.

**Run one recipe:**

```
python denoise/run.py experiments/tune_median.json --workers 4
```

It prints progress per frame and writes `outputs/tune_median/`. If that folder already has results it stops; add `--name my_test` to write somewhere new.

**Try a new filter setting:** copy a recipe, change the values in `grid`, give it a new `name`, and run it. Open `summary.csv` in Excel and compare `flat_mae_mm` and `band2_mae_mm` with the `raw` row.

**Rebuild everything:** the full order is in `README.md` under "Reproduce everything". The masked bilateral tuning takes about an hour; everything else runs in minutes.

**Put it on GitHub:** the folder is already a git repository with a `.gitignore`, and nothing is committed yet.

```
git add .
git commit -m "Kinect Lite phase 1: depth denoising"
git remote add origin <your repository URL>
git push -u origin main
```

## How to read the outputs

Start with `RESULTS/README.md`; it lists the pictures in reading order. For numbers, open a run's `summary.csv`: one row per setting, the `raw` row first as the reference.

**The columns that matter most in `summary.csv`:**

| column | meaning |
|---|---|
| `config` | the setting, e.g. `iamedian_k9_gate0.4` = invalid-aware median, 9x9 window, 0.4 m gate |
| `all_mae_mm` | mean error over all scored pixels (mm) |
| `flat_mae_mm` | mean error on flat areas, more than 5 px from an edge |
| `band2_mae_mm` | mean error within 2 px of an edge |
| `delta_vs_raw_median_mm` | typical change per frame vs raw; negative = better |
| `frames_worse_than_raw` | on how many frames the filter made things worse |
| `own_mae_mm_framemean` | error on the filter's own valid pixels, including any it filled |
| `coverage_pct_mean` | share of true-depth pixels that still have a value |
| `runtime_ms_median` | time per frame |

TUM runs have different columns: `change_flat_mm`, `change_edges_mm` (how much the depth changed), `flat_roughness_mm` (lower = smoother) and `holes_pct`.

**Reading the figures:**

- **Error images:** light = small error, dark = 5 cm or more. Dark outlines = edge errors.
- **Views from above:** grey = true surface, blue = points after filtering, camera at (0, 0). Points in front of a surface are errors.

- **Tuning curves:** each curve is one setting family; the dashed line is the raw input; lower is better.
- **3D viewers** (`RESULTS/3d_viewer_*.html`): open in a browser; drag to rotate, scroll to zoom, click a name in the legend to show or hide that cloud.

## Known issues and limits

The results are solid for ICL; how far they carry to a real sensor is not yet measured.

- **Simulated noise.** ICL's edge spikes likely come from how the noise was simulated. The medians' big edge advantage may shrink on real data.
- **1-pixel misalignment.** The noisy and true ICL images are about 1 px out of line, varying across the image. It adds about 1.5 mm to every flat-area score and more at edges. We did not correct it, so all filters carry it equally.
- **Same video for tuning and testing.** The frames are 25 apart in one sequence: fair, but not an independent test.
- **Selection by overall error.** Edge pixels dominate that number. Choosing by flat-area error would pick other settings; every tested setting is in `outputs/tune_*/summary.csv`.
- **No true depth on TUM.** On real data we show what changes, not what is right.
- **Bilateral tuned on ICL.** Its range setting ($\sigma r = 1$ m) is too wide for real edges, which is why it creates stray points on TUM.
- **Speed.** The masked bilateral (1.8 s per frame) and the invalid-aware median (0.2 s) are plain NumPy and not fast enough for live 30 fps use.
- **Plain median edge case.** It treats missing pixels as 0 m, so it can fill holes with wrong values and delete measured pixels.

## What comes next: ICP

ICP estimates how the camera moved between two depth frames by lining up their point clouds. Run on denoised depth and compared with TUM's true camera path, it becomes the real-data test the filters still need.

### Already in place

- Denoised depth: `denoise/filters.py` (start with the invalid-aware median, k 9, 0.4 m gate).
- Depth to 3D points: `denoise/pointcloud.py`, with both cameras' settings.

- True camera paths: `ICL NUIM/gt/livingRoom0.gt.freiburg` and TUM's `groundtruth.txt` (8,710 poses at 100 Hz; 2,507 of 2,509 depth frames have a pose within 20 ms).

**Plan**

1. Frame-to-frame ICP on ICL, checked against its true path.
2. The same on TUM, once each with raw, median and bilateral input.
3. Save each estimated path as `timestamp tx ty tz qx qy qz qw`, one line per frame.
4. Score with TUM's evaluation scripts offline: relative pose error over 1 s (drift, the main score for frame-to-frame tracking) and absolute trajectory error.
5. The filter with the lowest error on TUM is the real-data winner.

**Watch out for**

- ICL's pose file has 1,508 lines for 1,509 depth frames, numbered from 1; check which pose belongs to depth frame 0.
- TUM poses must be matched to depth frames by nearest timestamp.
- TUM has 16.5 % missing depth; ICP must skip empty pixels.
- Use the regular TUM sequence, not `_validation`: validation sequences have no ground truth and can only be scored by TUM's online form.

## Glossary

| term | meaning |
|---|---|
| depth image | a picture where each pixel holds a distance instead of a colour |
| point cloud | the depth pixels turned into 3D points with the camera model |
| intrinsics | the camera numbers fx, fy, cx, cy that turn pixels into 3D directions |
| ground truth (GT) | the true value: true depth on ICL, the true camera path on TUM |
| MAE | mean absolute error: the average size of the depth error, in mm |

| term | meaning |
|---|---|
| flat area | pixels more than 5 px from any depth edge in the true image |
| edge band | pixels within 2 px of a depth edge in the true image |
| matched mask | the pixels scored for every filter: valid in the true depth, the input and every output |
| own MAE | error on a filter's own valid pixels, including any it filled |
| 0.4 m gate | treating depth below 0.4 m as missing before filtering |
| masked filter | a filter that leaves missing pixels out of its average instead of counting them as 0 m |
| invalid-aware median | a median over the valid pixels of the window only; holes stay holes |
| σ (sigma) | the width of a Gaussian, in pixels; for the bilateral, σr is a depth difference in metres |
| tuning / test frames | frames used to pick settings (0, 50, 100 …) vs frames used to score them (25, 75, 125 …) |
| recipe | an experiment file in `experiments/` |
| ICP | Iterative Closest Point: estimates camera motion by aligning two point clouds |
| ATE / RPE | absolute trajectory error / relative pose error: how far an estimated camera path is from the true one |